AMSS 2026 — Lab 4: Critique Session — Behavioral Artifacts

Traian-Florin Șerbănuță

2026

Lab 4: Red-Team the Behaviour

Three phases, 100 minutes — **teams of 3-5. Same format as Lab 3.**

1. **Brief + rubric** (~10 min) — the domain, the three artifacts, the read-order.
2. **Defect hunt** (~60 min) — hunt every planted defect in three flawed behavioural artifacts, rating each by severity.
3. **Defect-hunt-off** (~30 min) — reveal ground truth, score, crown the sharpest critics.

Deliverable: a **severity-rated defect log** per team, committed to the course lab repo.

The Flip: From Structure to Behaviour

- **Lab 3:** you red-teamed flawed **structural** artifacts — class, package, component.
- **Today:** flawed **behavioural** artifacts — sequence, state, activity.

Same architect-and-critic loop, all critic. No AI to drive — pure reading and critique, against the clock.

Before You Start

- Form teams of **3-5**. One member is the **scribe** (owns the defect log + the commit).
- You receive at lab start: your **team-id**, the lab-repo URL, and the three artifacts (in `lab04/README.md` on clone).
- All three artifacts model the **same** system — the ATM below.

The Domain: ATM

The artifacts model an ATM. The **domain is honest** — every defect is in the artifacts, not the description.

- A customer **inserts a card** and enters a **PIN**; after 3 wrong PINs the card is **retained**.
- Once verified, the customer can **withdraw cash**: the ATM checks the balance with the **bank**, dispenses if funds suffice, and **ejects** the card.
- On **insufficient funds** or **cancel**, no cash is dispensed and the card is ejected.
- The ATM **prints a receipt** and returns to idle after each session (or on inactivity timeout).

The Three Artifacts

All three describe the same ATM, and **all three are flawed**:

1. A **sequence diagram** — the messages for "withdraw cash".
2. A **state machine** — the card/session lifecycle.
3. An **activity diagram** — the withdrawal workflow.

Find every planted defect. Rate each. The team that scores highest wins the hunt-off.

Your Rubric: the Read-Orders

Sequence (Week 6):

1. Each **message** maps to a real responsibility?
2. **Order** causally possible? (no result before its cause)
3. **Returns** present? (no dangling calls)
4. **Failure path** modelled? (not just the happy path)

State machine (Week 7):

5. Every state **reachable**? Every state **escapable** (or final)?
6. **Guards** wherever one event branches? Initial present?

Activity (Week 7):

7. Every **decision merges**? Every **fork joins**? A path to a **final**?

The Defect Vocabulary

Name each defect with the catalogue from Weeks 6-7:

- **Sequence:** fabricated message · impossible order · missing return · only the happy path · message to a stranger / invented lifeline.
- **State:** orphan / unreachable state · dead-end state · missed guard · no initial / final.
- **Activity:** missing merge · fork without join · no final · decision with no condition.

A defect named with the catalogue scores; a vague "this looks off" does not.

Severity — Rate Every Defect

- **High (3)** — breaks the behaviour or propagates downstream: an orphan/dead-end state, a missing failure path, cash dispensed before authorisation.
- **Medium (2)** — wrong but local: a fabricated message, a missed guard, a missing final.
- **Low (1)** — cosmetic or clarity: an invented log lifeline, an untriggered transition.

Every logged defect needs a severity. Rating *is* the skill.

Phase 2: The Defect Hunt (60 min)

As a team, walk each artifact with the read-order. For **every** defect, log:

- **Artifact** (sequence / state / activity)
- **Defect name** (from the catalogue) + **where** (the message, state, or node)
- **Severity** (high / med / low)
- **One line: why it matters**

Divide and conquer, or sweep together — but converge on one shared log.

How the Hunt-Off Is Scored

Calibrate — precision counts (same rules as Lab 3):

- **Correct defect** scores its severity weight (high 3 / med 2 / low 1).
- **Severity** must be within one band for full credit; off by two bands loses a point.
- **False positive** — a "defect" that isn't real — costs a point (minus 1).
- No double-counting the same defect under two names.

Spraying guesses backfires. Find the real defects, rate them honestly.

The Defect Log (Deliverable)

One table, all three artifacts, on branch `lab04/<team-id>`:

Artifact	Defect (catalogue name)	Where	Severity	Why it matters
sequence	only happy path	no insufficient-funds branch	high	the costly case is unmodelled
state	orphan state	`Blocked`	high	nothing ever reaches it
activity	fork without join	dispense / receipt	high	flows never synchronise

Phase 3: Defect-Hunt-Off + Submit (30 min)

- We reveal ground truth **artifact by artifact**; each team scores its own log live.
- Each team nominates its **best catch** (highest-severity real defect) — bonus point.
- Highest net score wins.

```
git checkout -b lab04/<team-id>
mkdir -p lab04/<team-id>
git add lab04/<team-id>/defect-log.md
git commit -m "Lab 4: <team-id> ATM defect log"
git push -u origin lab04/<team-id>
```

Grading — Pass / Redo

Pass needs both:

1. `defect-log.md` committed by the team by the deadline.
2. The log names **at least five** real defects across the three artifacts, each with a severity and a one-line reason, **including at least one high**.

The competition is for sharpness and bragging rights; the gate is an honest, catalogue-named log. We grade your critique, not your speed.

Why This Matters

You have now red-teamed both **structure** (Lab 3) and **behaviour** (Lab 4) — naming defects and rating them cold is exactly the **Critique** skill the oral defense checks. Flawed behaviour propagates to tests and code.

Next: **Week 9** deepens patterns; the labs now turn to **your own project** — **Lab 5** is the checkpoint defense.